Unidad 6: Interfaces Serie de Sensores Digitales

Clase 3: El Bus SPI (Serial Peripheral Interface)

1. Introducción: La Necesidad de Velocidad y Sincronización

El bus **SPI (Serial Peripheral Interface)** fue desarrollado por Motorola como la solución de comunicación serie síncrona y de alta velocidad por excelencia[cite: 1]. Es el protocolo preferido para componentes que requieren la máxima tasa de transferencia de datos (tarjetas SD, displays gráficos de alta resolución, memorias flash), ya que opera en modo **Full-Duplex**, permitiendo enviar y recibir información simultáneamente[cite: 1].

A diferencia de I^2C, que gestiona el tráfico mediante direccionamiento de software, el SPI utiliza un control de línea física directa para la selección de dispositivos[cite: 1]. Esto elimina la sobrecarga de protocolo (*overhead*) y permite alcanzar velocidades superiores a los **50 MHz**[cite: 1].

2. Topología y Componentes Clave

El SPI opera bajo una topología estricta de **Maestro Único y Múltiples Esclavos**[cite: 1]. Requiere cuatro líneas principales de comunicación[cite: 1]:

SCLK (Serial Clock): Generado exclusivamente por el Maestro para sincronizar la lectura de cada bit[cite: 1].

MOSI (Master Out Slave In): Canal de salida del Maestro y entrada a los Esclavos[cite: 1].

MISO (Master In Slave Out): Canal de salida de los Esclavos y entrada al Maestro[cite: 1].

SS / CS (Slave Select / Chip Select): Línea individual. Se pone en **LOW (0V)** para activar un esclavo específico[cite: 1].

> **La Mecánica del Intercambio (Full-Duplex):**
> El SPI funciona como dos registros de desplazamiento conectados en anillo[cite: 1]. Por cada pulso de reloj `SCLK`, se envía un bit por `MOSI` y se recibe simultáneamente un bit por `MISO` [cite: 1]. No existe transmisión sin recepción[cite: 1].

3. Detalle Técnico Explicado: Los Modos SPI (CPOL y CPHA)

Para que el Maestro y el Esclavo se entiendan, deben acordar cómo usar la señal de reloj `SCLK` [cite: 1, 2]. Esto se configura mediante dos parámetros fundamentales[cite: 2]:

A. CPOL (Clock Polarity — Polaridad del Reloj)

Responde a la pregunta: *¿En qué nivel lógico descansa la línea SCLK cuando NADIE está transmitiendo (en reposo)?*[cite: 2]

- **CPOL = 0:** El reloj descansa en **LOW (0 Volts)**[cite: 2]. El primer pulso será una subida ($\uparrow$).

- **CPOL = 1:** El reloj descansa en **HIGH (5V / 3.3V)**[cite: 2]. El primer pulso será una bajada ($\downarrow$).

B. CPHA (Clock Phase — Fase del Reloj)

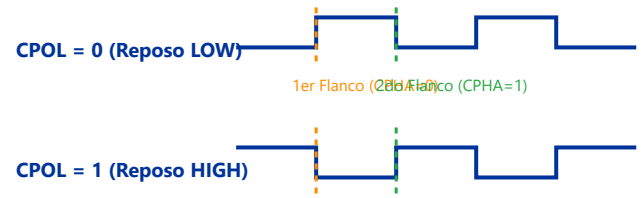
Responde a la pregunta: *¿En cuál flanco o borde del reloj se "saca la foto" (muestrea) para leer el dato?*[cite: 2]

- **CPHA = 0:** El dato se lee en el **PRIMER flanco** de cambio de voltaje (sin importar si es subida o bajada) [cite: 2].
- **CPHA = 1:** El dato se lee en el **SEGUNDO flanco** de cambio de voltaje (el flanco de retorno)[cite: 2].

Tabla de Combinación de Modos SPI:[cite: 2]

Modo SPI	CPOL	CPHA	Estado de SCLK en Reposo	Flanco donde se Muestrea (Lee) el Dato
Modo 0	0	0	LOW (0V)	**Primer flanco:** Flanco de Subida ($\uparrow$)
Modo 1	0	1	LOW (0V)	**Segundo flanco:** Flanco de Bajada ($\downarrow$)
Modo 2	1	0	HIGH (VCC)	**Primer flanco:** Flanco de Bajada ($\downarrow$)
Modo 3	1	1	HIGH (VCC)	**Segundo flanco:** Flanco de Subida ($\uparrow$)

Esquema Visual de Muestreo según CPOL y CPHA



4. Análisis Comparativo y Escalabilidad

Aunque el protocolo SPI es sumamente rápido, su principal desventaja radica en el uso de pines[cite: 2]. Para conectar N esclavos, se requieren $3 + N$ pines digitales en el Maestro[cite: 2].

Protocolo	Pines Necesarios (para 5 Esclavos)	Velocidad Típica Máxima	Mecanismo de Control
UART	10 (5 RX / 5 TX)[cite: 2]	~1 Mbps[cite: 2]	Baudrate configurado previamente[cite: 2]
I^2C	2 (SDA, SCL)[cite: 2]	3.4 Mbps[cite: 2]	Direccionamiento por Software (7/10 bits)[cite: 2]
SPI	8 (3 compartidos + 5 CS)[cite: 2]	> 50 Mbps[cite: 2]	Selección Física por línea CS[cite: 2]

5. Actividades Prácticas y Comparativas

Actividad 1: Cuadro Comparativo de Protocolos (30 min)

Completa en tu cuaderno la siguiente síntesis comparativa entre los tres buses estudiados[cite: 3]:

Característica	UART	I^2C	SPI
Topología	Punto a Punto[cite: 3]	Multi-Maestro / Multi-Esclavo[cite: 3]	Maestro Único / Multi-Esclavo[cite: 1]
Pines Mínimos	2 (TX, RX)[cite: 3]	2 (SDA, SCL)[cite: 3]	3 + CS por esclavo[cite: 2]
Sincronización	Asíncrona (Sin reloj)[cite: 3]	Síncrona (Reloj SCL)[cite: 3]	Síncrona (Reloj SCLK)[cite: 1]
Velocidad	Baja (<1 Mbps)[cite: 3]	Baja/Media (<3.4 Mbps)[cite: 3]	Muy Alta (>50 Mbps)[cite: 1, 2]
Modo de Flujo	Full-Duplex[cite: 3]	Half-Duplex[cite: 3]	Full-Duplex[cite: 1, 3]
Direccionamiento	No requiere[cite: 3]	Por Software (7/10 bits)[cite: 3]	Por Hardware (Línea CS)[cite: 1]

Actividad 2: Modelado del Bus SPI en Arduino

Objetivo: Comprender la lógica de selección por CS y la transmisión simultánea Full-Duplex usando LEDs indicadores[cite: 3].

Conexiones del Circuito:

- Pin 8: Conectado a LED 1 (Simula la línea `CS`)[cite: 3]
- Pin 9: Conectado a LED 2 (Simula la línea `MOSI`)[cite: 3]
- Pin 10: Conectado a LED 3 (Simula la línea `MISO`)[cite: 3]

Código Arduino Completo:[cite: 4, 5]

```
// Definición de Pines
#define CS_PIN   8   // Chip Select (Selector de Esclavo)
#define MOSI_PIN 9   // Master Out Slave In (Salida Maestro)
#define MISO_PIN 10  // Master In Slave Out (Entrada Maestro)

void setup() {
  Serial.begin(9600);
  Serial.println("====================================");
  Serial.println("  SIMULACION DEL BUS SPI INICIADA   ");
  Serial.println("====================================");

  // Configuración de Pines como Salida
  pinMode(CS_PIN, OUTPUT);
  pinMode(MOSI_PIN, OUTPUT);
  pinMode(MISO_PIN, OUTPUT);

  // Estado Inicial: Bus Inactivo / Esclavo Deshabilitado
  digitalWrite(CS_PIN, HIGH);
  digitalWrite(MOSI_PIN, LOW);
  digitalWrite(MISO_PIN, LOW);
}

void loop() {
  // PASO 1: Esclavo Desconectado (CS HIGH)
  Serial.println("\n------------------------------------");
  Serial.println("PASO 1: Esperando. Esclavo Desconectado (CS HIGH)");
  delay(2000);

  // PASO 2: CONEXIÓN - Habilitar el Esclavo (CS LOW)
  digitalWrite(CS_PIN, LOW);
  Serial.println("PASO 2: CONEXIÓN! Esclavo Seleccionado (CS LOW)");
  delay(1000);

  // PASO 3: INTERCAMBIO DE DATOS (Full-Duplex)
  // Encendemos MOSI y MISO simultáneamente
  digitalWrite(MOSI_PIN, HIGH);
  digitalWrite(MISO_PIN, HIGH);
  Serial.println("PASO 3: Intercambiando datos (Full-Duplex)");
  Serial.println("--> MOSI y MISO (LEDs) están en HIGH simultáneamente.");
  delay(3000);

  // Apagamos líneas de datos
  digitalWrite(MOSI_PIN, LOW);
  digitalWrite(MISO_PIN, LOW);

  // PASO 4: DESCONEXIÓN - Deshabilitar el Esclavo (CS HIGH)
  digitalWrite(CS_PIN, HIGH);
  Serial.println("PASO 4: Desconectando. Esclavo Deshabilitado (CS HIGH)");
  delay(5000);
}
```